

Physical Design of RISC-V based System-on-Chip using OpenLane

Muhammad Fauzan[‡], Raihan Fadhil Yanuarsyah[‡], Muhammad Hanif Hibatullah[‡],
Radithya Arisaputra[‡], Infall Syafalni^{‡§*}, Nana Sutisna^{‡§}, Trio Adiono^{‡§}, and Makoto Ikeda^{*†}

[‡]School of Electrical Engineering and Informatics, Bandung Institute of Technology, Indonesia

[§]University Center of Excellence on Microelectronics, Bandung Institute of Technology, Indonesia

^{*}System Design Laboratory, School of Engineering, The University of Tokyo, Tokyo, Japan

[†]Graduate School of Engineering, The University of Tokyo, Tokyo, Japan

Abstract—Open-source tools for designing integrated circuits are crucial for electrical engineering students. However, more references are necessary to promote the tools to the academic environments. This paper presents design and implementation of a PicoSoC (Pico System-on-Chip) using the OpenLane automated VLSI flow. The PicoSoC comprises several modules, including the PicoRV32 CPU, UART and SPI peripherals, and memory blocks. The design methodology encompasses RTL simulation, macro hardening, core hardening, full-chip hardening, and gate-level simulation. The results indicate successful physical design and functional verification, demonstrating that the gate-level simulation outputs align with those from RTL simulation. The work is useful for designing the physical layout of digital circuits, SoC architecture development, and also promoting open-source IC design tools in academic environments.

Index Terms—PicoSoC, OpenLane, PicoRV32, RTL Simulation, Gate Level Simulation, VLSI Design, Physical Design

I. INTRODUCTION

Open-source tools for designing integrated circuits [1] are crucial for academics and industries. In academics, lecturers are able to teach the students complete design tools using open-source tools. Moreover, more innovations are expected to be produced due to the flexibility of the licenses. Recently, Google has developed an open-source tool for digital IC design called OpenLane [2].

Some researchers already implementing IC using OpenLane both for research or education *i.e.*, in [3], [4], [5] and [6]. Moser et al. in [3] have implemented a low-power 12-bit non-binary charge-redistribution SAR-ADC on SKY130 PDK OpenLane. The design combines analog and digital circuits through macros. Shah et al. in [4] has implemented mini FPGA with 896 total LUTs, 32×4 DFFRAM with two read ports, and 1 column of double height DSP supertiles, each comprising of 8×8 multiply and 20-bit accumulate. Hesham et al. in [6] has implemented RISC-V based processor "DwarfRV32" on OpenLane and commercial tools to compare its parameters like timing performance and power comparison.

In this paper, we present design and implementation of a PicoSoC (Pico System-on-Chip) using the OpenLane automated VLSI flow. The PicoSoC comprises several modules, including the PicoRV32 CPU, UART and SPI peripherals, and memory blocks. The design methodology encompasses RTL simulation, macro hardening, core hardening, full-chip hardening,

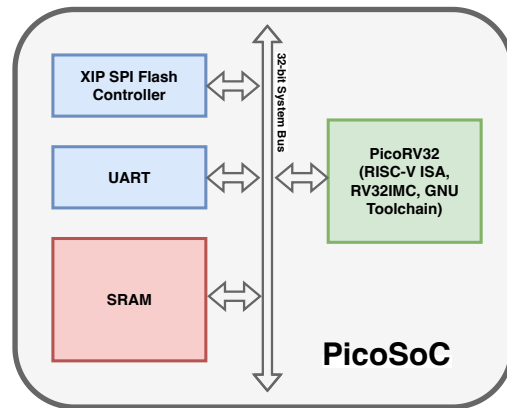


Fig. 1: Proposed PicoSoC Block Diagram

ing, and gate-level simulation. The results indicate successful physical design and functional verification, demonstrating that the gate-level simulation outputs align with those from RTL simulation.

TABLE I: PicoSoC Memory Map

Address Range	Size	Description
0x00000000 – 0x00FFFFFF	1 MB	Internal SRAM
0x01000000 – 0x01FFFFFF	1 MB	External Serial Flash
0x02000000 – 0x02000003	4 Bytes	SPI Flash Controller Config Register
0x02000004 – 0x02000007	4 Bytes	UART Clock Divider Register
0x02000008 – 0x0200000B	4 Bytes	UART Send/Recv Data Register
0x03000000 – 0xFFFFFFFF	3.99 GB	Memory Mapped User Peripherals

II. RISC-V-BASED SOC IMPLEMENTATION USING OPENLANE

A. Overall Proposed Architecture

The proposed system-on-chip architecture with PicoRV32 CPU can be seen in Figure 1. PicoSoC contains PicoRV32 as

its CPU, UART and SPI as its peripherals, and SRAM as its memory. PicoSoC has 32-bit address space so we can access the address with 4GB length. The memory map of PicoSoC can be seen in Table I.

B. Macro Setting for All Circuit Components

The configuration to create the macros that make up the System-on-Chip using OpenLane [2] is stored in the configuration variables in the config.tcl file. Configurations that must be specified are the name of the top-level module of the design, the path of the design's Verilog file, the clock period, the name of the Net input to the root clock buffer used in Clock Tree Synthesis, and the name of the design clock port used in Static Timing Analysis [2].

TABLE II: General Configuration Variables

Variable	Description
DESIGN_NAME	The name of the top-level module of the design
PDK	Specifies the process design kit (PDK)
VERILOG_FILES	The path of the design's verilog files
CLOCK_PERIOD	The clock period for the design in ns
CLOCK_PERIOD	The name of the design's clock port used in Static Timing Analysis.
CLOCK_NET	The name of the Net input to root clock buffer used in Clock Tree Synthesis.

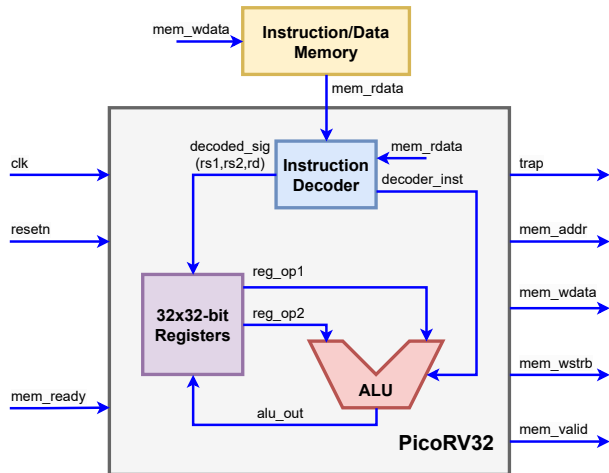


Fig. 2: PicoRV32 Simplified Block Architecture [1].

Other configurations that can be set are synthesis, floorplanning, placement, and signoff. Synthesis configuration specifies the synthesis process, some of which handles clock uncertainty, flattens the design hierarchy, optimizes resource usage, and selects the type of adder used in the design. The placement configuration specifies core utilization, die size and aspect ratio, power, and ground network definition. The placement and routing configuration specifies placement density, optimization

strategies during placement, clock tree synthesis, routing core utilization, and congestion handling in the routing phase. The signoff and verification configuration specifies specific signoff checks such as comprehensive verification and IR drop analysis [2].

TABLE III: Floorplanning and Physical Design Configuration Variables

Variable	Description
FP_CORE_UTIL	The core utilization percentage.
FP_ASPECT_RATIO	The core's aspect ratio (height/width)
FP_SIZING	To determine the size of the macros or the core. We can adjust it with "relative" or "absolute".
DIE_AREA	Specific die area to be used in floor planning.
VDD_NETS	Specifies the power nets/pins to be used when creating the power grid for the design.
GND_NETS	Specifies the ground nets/pins to be used when creating the power grid for the design.
FP_PDN_CORE_RING	Sets whether the core ring for PDN (Power Distribution Network) is enabled.
FP_PDN_MULTILAYER	Sets whether PDN uses multiple layers.

TABLE IV: Synthesis Configuration Variables

Variable	Description
SYNTH_CLOCK_UNCERTAINTY	Specifies a value for the clock uncertainty in the pre-CTS stages.
SYNTH_NO_FLAT	A flag that disables flattening the hierarchy during synthesis, only flattening it after synthesis, mapping and optimizations.
SYNTH_SHARE_RESOURCES	A flag that enables Yosys to reduce the number of cells by determining shareable resources and merging them.
SYNTH_ADDER_TYPE	Adder type to which the <i>addandsb</i> operators are mapped to.
SYNTH_USE_PG_PINS_DEFINES	A flag to define the use of power and ground pins.

C. PicoRV32

PicoRV32 is a size-optimized open-source RISC-V core. The CPU only uses 750-200 LUTs on FPGAs with Xilinx's 7-series architecture with a maximum frequency of 250-450 MHz. In addition, PicoRV32 has a selectable interface between native memory and AXI4-lite master. In this case, the native memory interface is used to connect with the memory module in this SoC [7]. Figure 2 shows a simplified main block diagram of the PicoRV32, where the main blocks are the instruction decoder, registers, and ALU.

TABLE V: Placement and Routing Configuration Variables

Variable	Description
PL_TARGET_DENSITY	The desired placement density of cells.
PL_BASIC_PLACEMENT	Specifies whether the placer should run basic placement or not (by running initial placement, increasing the minimum overflow to 0.9, and limiting the number of iterations to 20).
RUN_CTS	Enable clock tree synthesis
ROUTING_CORES	Specifies the number of threads to be used in TritonRoute.
GLB_RT_ALLOW_CONGESTION	Allow congestion in the resulting guides.

TABLE VI: Signoff and Verification Configuration Variables

Variable	Description
RUN_CVC	Runs CVC on the output spice, which is a Circuit Validity Checker.
RUN_IRDROP_REPORT	Creates an IR Drop report using OpenROAD PSM.

D. UART

UART peripheral is implemented as Figure 3. The module is instantiated in the top module with a configurable BAUD rate through `reg_div_we` and `reg_div_di` ports. BAUD rate can be read through `reg_div_do` port. The TX data is sent from the CPU through `reg_dat_we` and `reg_dat_di` ports. RX data is received by the CPU through `reg_dat_do` and `reg_dat_wait` is used for the wait flag. UART communicates with the external system through `ser_tx` and `ser_rx` ports. The address space of UART can be seen in Table I.

E. SPI

SPI peripheral is implemented as Figure 4. The module is instantiated in the top module and controlled through address space in Table I.

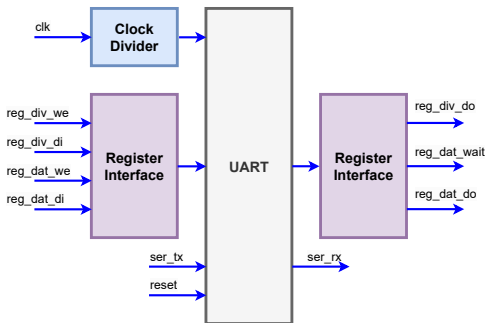


Fig. 3: Proposed UART Block Diagram

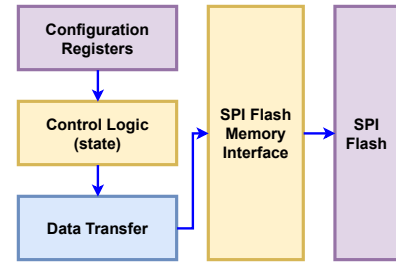


Fig. 4: Proposed SPI Block Diagram

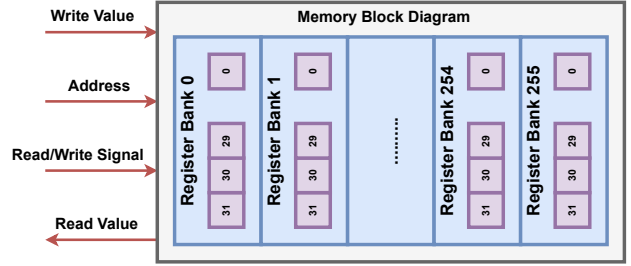


Fig. 5: Proposed Memory Block Diagram

F. Memory

The memory module synthesized into a gate-level netlist comes from the Verilog code that describes the register [8]. So that the memory implemented is a register composed of DFF as shown in Figure 5. With a word size of 256 and each register having a width of 32 bits, 256 registers of 32 bits will be synthesized. This approach produces a very large memory block, in order to produce memory with a more optimal size, the memory module can use standard cell SRAM from SKY130 technology.

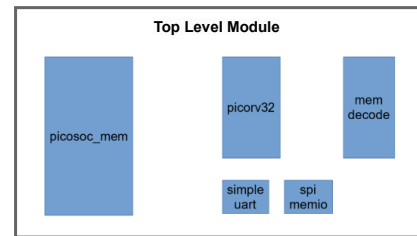


Fig. 6: Proposed Floorplanning using OpenLane

G. Memory Decoder

In order for the layout to be organized and prevent routing congestion, the physical implementation method is done without standard cells at the top level [9]. Therefore, the logic operation in the top-level Verilog code, `picosoc`, needs to be moved into a separate module, `memory decode` as it functions as a decoder [8].

III. EXPERIMENTAL RESULTS: TOP LEVEL INTEGRATION

A. Floorplanning and Settings

To combine all of the macros, it takes hardened GDS, LEF, and lib macro files and specifies the location of the files in

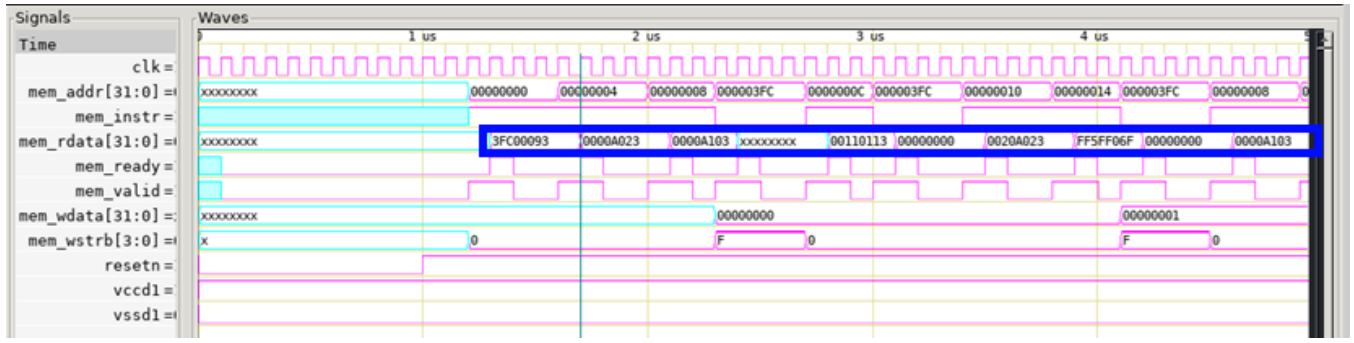


Fig. 7: PicoRV32 CPU Post-Layout Simulation

config.tcl. The files are placed in the GDS, LEF, and LIB folders. The top-level Verilog file picosoc is placed in the src folder. Then, the hardened gate-level verilog macro files are placed in the verilog_blackbox folder.

The next step is floorplanning to determine the position of macros. In general, the following is the floorplanning that will be implemented. The design of the floorplanning of the core can be seen in Figure 6. The basic configuration needed to do floorplanning can be seen in Table II and III. The size and the placement of each macro can be seen in Table VII.

TABLE VII: Die Area and Coordinate of Each Macro

Macro	Die Area (μm)	Coordinate (North)
PicoRV32 (CPU)	600×600	1400 600
UART	200×200	1400 200
SPI	200×200	1800 200
Memory Decoder	200×600	2200 600
Memory	930×940	200 200

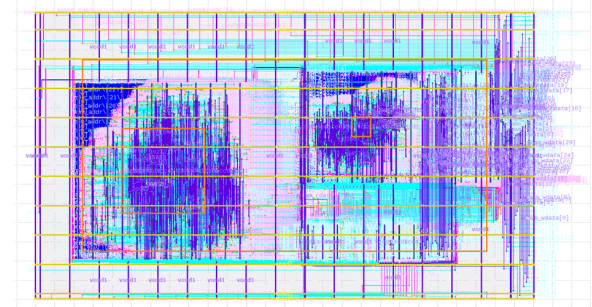
B. Post Layout Simulation

For post-layout simulation, all macro simulations for post-layout were successfully executed. Figure 7 shows the post-layout simulation of PicoRV32. From the simulation, the intended instructions are fulfilled as follows; 32'h3fc0093 (*li, x1, 1020*), 32'h0000a023 (*sw, x0, 0(x1)*), 32'h0000a103 (*loop: lw, x2, 0(x1)*), 32'h0011a113 (*addi, x2, x2, 1*), 32'h0020a023 (*sw, x2, 0(x1)*), and 32'hff5ff06f (*j, < loop >*).

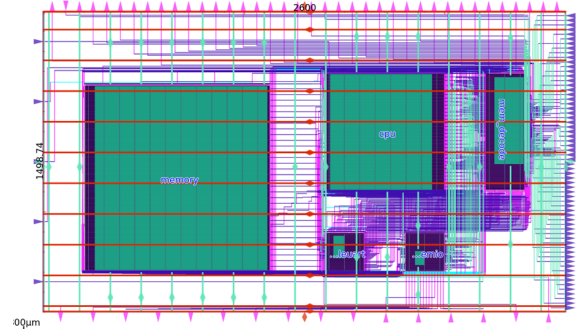
To find the maximum frequency, examine the values for `finish_timing_setup_ws` or `report_worst_slack` in the final report [10]. "Slack" refers to the difference between the timing constraint (100 ns) and the actual signal propagation time. Positive slack indicates that the constraint is satisfied (*i.e.*, there is extra time available), while negative slack indicates that the constraint is not met. Using the slack, the frequency is calculated as $Frequency_{max} = 1/(constraint - slack)$. In this case, the max frequency is $1/(100ns - 47.14ns) \approx 19$ MHz.

C. Place and Route Results

Figures 8a and 8b are the results of the picosoc layout viewed using OpenROAD and KLayout.



(a) PicoSoC Layout seen with KLayout



(b) PicoSoC Layout seen with OpenROAD

Fig. 8: PicoSoC Layout Results

IV. CONCLUSION

In this paper, RISC-V-based PicoSoC System on Chip is used as a co-processor. The SoC consists of CPU, Memory, Memory Decoder, and interfaces such as UART and SPI. The SoC components are first implemented in physical design as macros. These macros will be used to implement the Core design, namely SoC. Creating a Macro using OpenLane requires a Verilog source file, pin order, and config containing setting variables to run OpenLane. Creating Macro Integration into Core requires additional source files, namely GDS, LEF, LIB, and Verilog Gate Level macros used. All Macros were successfully created and the pre-layout and post-layout simulations were the same. The work is useful for designing the physical layout of digital circuits, SoC architecture development, and also promoting open-source IC design tools in academic environments.

ACKNOWLEDGEMENT

This work is supported by 2023 International Research Program provided by LPPM, Bandung Institute of Technology under grant no. [LPPM.PN-16-22-2023].

REFERENCES

- [1] G. Kanda and K. Ryoo, "Design of an integrated cryptographic soc architecture for resource-constrained devices," in *International Journal of Electrical and Electronics Research*, vol. 10, 2022, pp. 230–244.
- [2] M. Shalan and T. Edwards, "Building openlane: A 130nm openroad-based tapeout- proven flow : Invited paper," in *2020 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*, 2020, pp. 1–6.
- [3] M. M. Z. G. Fath, Patrick and H. Pretl, "Open-source design of integrated circuits," in *e+i Elektrotechnik und Informationstechnik*, vol. 141, 2024, p. 76–87.
- [4] M. Shah, J. Ternes, and D. Koch, "Fabulous demo: Open source fpga on sky130," in *2023 33rd International Conference on Field-Programmable Logic and Applications (FPL)*. Los Alamitos, CA, USA: IEEE Computer Society, sep 2023, pp. 365–365. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/FPL60245.2023.00070>
- [5] Z. Li and A. C. Carusone, "Design and optimization of low-dropout voltage regulator using relational graph neural network and reinforcement learning in open-source sky130 process," in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, 2023, pp. 01–09.
- [6] S. Hesham, M. Shalan, M. W. El-Kharashi, and M. Dessouky, "Digital asic implementation of risc-v: Openlane and commercial approaches in comparison," in *2021 IEEE International Midwest Symposium on Circuits and Systems (MWSCAS)*, 2021, pp. 498–502.
- [7] M. Milanovic, "Picrov32 - a size-optimized risc-v cpu," 2019. [Online]. Available: <https://github.com/YosysHQ/picrov32>
- [8] M. B. Aykenar, "Openlane designs," 2024. [Online]. Available: <https://github.com/mbaykenar/openlane-designs>
- [9] M. Venn, "Caravel wishbone demo," 2023. [Online]. Available: https://github.com/mattvenn/wishbone_buttons_leds
- [10] T. O. Project, "Micro 2022 tutorial," 2022. [Online]. Available: <https://github.com/The-OpenROAD-Project/micro2022tutorial/tree/main>